



OPEN ACCESS

EDITED BY

Haoxuan Dong,
National University of Singapore,
Singapore

REVIEWED BY

Pavel Loskot,
The Zhejiang University-University of
Illinois at Urbana-Champaign Institute,
United States
Xinyu Gao,
Nanjing University, China

*CORRESPONDENCE

Chinyere Grace Kennedy,
✉ gkennedy@kduniv.ac.kr
Mukiibi Moses,
✉ mosesmukiibi21@gmail.com
Simeon Okechukwu Ajakwe,
✉ profsimeon@kduniv.ac.kr

RECEIVED 30 June 2026

REVISED 17 August 2026

ACCEPTED 20 August 2026

PUBLISHED 10 September 2026

CITATION

Kennedy CG, Moses M and Ajakwe SO
(2026) Explainable and cyber-resilient
vision transformers for joint steering and
collision-avoidance perception in
autonomous drone navigation: a multi-
task Grad-CAM framework.
Front. Future Transp. 7:1923890.
doi: 10.3389/ffutr.2026.1923890

COPYRIGHT

© 2026 Kennedy, Moses and Ajakwe. This
is an open-access article distributed under
the terms of the [Creative Commons
Attribution License \(CC BY\)](#). The use,
distribution or reproduction in other
forums is permitted, provided the original
author(s) and the copyright owner(s) are
credited and that the original publication
in this journal is cited, in accordance with
accepted academic practice. No use,
distribution or reproduction is permitted
which does not comply with these terms.

Explainable and cyber-resilient vision transformers for joint steering and collision-avoidance perception in autonomous drone navigation: a multi-task Grad-CAM framework

Chinyere Grace Kennedy*, Mukiibi Moses* and
Simeon Okechukwu Ajakwe*

Department of Smart Computing, Kyungdong University, Goseong, Gangwon, Republic of Korea

Micro aerial vehicles (MAVs) flying through indoor corridors and other GNSS-denied environments cannot rely on satellite positioning, so a single monocular RGB camera is often the only sensing modality available. The navigation objective addressed in this paper is reactive, map-free corridor following: from each camera frame the onboard perception stack must choose a discrete steering direction that keeps the vehicle advancing along the corridor while avoiding walls and obstacles, rather than tracking a pre-specified waypoint, route, or global path. Prior work (DroNet, PULP-DroNet) addresses this problem with lightweight convolutional networks that map camera frames directly to a steering command and a collision flag; this paper asks whether a Vision Transformer can perform the same mapping with higher accuracy while remaining auditable. We present ViTDrone, an explainable artificial intelligence (XAI) framework that couples a Vision Transformer (ViT-Base/16) backbone with multi-task learning and gradient-weighted class activation mapping (Grad-CAM). From a single shared backbone the model simultaneously predicts a three-class steering command (left/forward/right) and a binary collision label. We build a scalable data pipeline that aggregates 309 acquisition sequences of the PULP-DroNet v3 benchmark, yielding 195,455 training and 76,390 test frames annotated with yaw-rate commands and collision risk. After five epochs of AdamW fine-tuning from ImageNet-21k weights, ViTDrone attains 76.35% yaw classification accuracy and 60.77% collision accuracy. To make the model auditable we adapt Grad-CAM to the transformer with a reshape transform over the patch tokens. The resulting saliency maps are reassuring: the network looks where a human pilot would, at floor-wall boundaries when it decides to turn and at overhead structure when it senses a collision. However, aggregate accuracy conceals substantial variation across classes. A per-class analysis, combining loss curves, confusion matrices, and a comparison against convolutional baselines, shows that the reported accuracies are driven disproportionately by the two majority classes (forward steering and no-collision), while minority classes, sharp turns and imminent collisions, are recognized far less reliably. We attribute this shortfall to two measurable causes rather than to insufficient backbone capacity: severe label imbalance (78.6% forward, 60.7% no-collision) and a persistent gap between training and validation loss that is consistent with distribution shift across acquisition sequences. Relative to prior work, which has evaluated either

convolutional navigation policies without explainability or Vision Transformer explainability outside the UAV navigation setting, this paper contributes (i) to our knowledge, the first integration of transformer-native Grad-CAM explainability with a multi-task steering-and-collision policy for MAV navigation, and (ii) a systematic diagnosis showing that class imbalance and domain shift, rather than model capacity, are the dominant factors limiting both accuracy and interpretability in this setting, together with a concrete research agenda for addressing them.

KEYWORDS

autonomous drone navigation, collision avoidance, explainable AI, GNSS-denied navigation, grad-CAM, micro aerial vehicles, multi-task learning, PULP-dronet

1 Introduction

Autonomous micro aerial vehicles (MAVs) have moved rapidly from research prototypes to operational platforms in precision agriculture, infrastructure and asset inspection, warehouse logistics, and search-and-rescue (Singla and Kaur, 2021). Their small form factor, low cost, and agility make them attractive for missions that are dangerous or impractical for human operators or larger aircraft. Yet the same constraints that make MAVs attractive also make their perception problem hard: limited payload restricts sensing to lightweight monocular cameras, limited compute restricts the size of on-board models, and the three-dimensional, cluttered environments in which MAVs operate demand fast, robust decisions. A modern UAV perception stack must therefore solve two intertwined problems in real time: where to go, i.e., generating a navigation command, and what to avoid, i.e., detecting imminent collisions.

While the cyber-resilience of connected aerial systems is frequently framed around mitigating external network disruptions and communication faults, true vehicular autonomy demands that the baseline perception engine is first resilient to internal, data-driven vulnerabilities. Accordingly, we argue that a safety-critical visuomotor policy cannot achieve systemic reliability unless it is inherently robust against structural ‘nuisances’ such as severe data imbalance, spatial distribution shifts, and environmental domain gaps.

A connected aerial vehicle faces a spectrum of external threats: adversarial patches injected into the visual field, GPS spoofing, and communication or link faults between the vehicle and its network, each of which has attracted its own dedicated defences (Ajakwe and Kim, 2024). But every one of those defences quietly assumes a perception engine that already behaves predictably on benign inputs. If the onboard vision stack collapses under ordinary internal nuisances, class imbalance, spatial distribution shift, an environmental domain gap, then there is no stable foundation on which network-level or anti-spoofing resilience can be built; the attacker need not even reach the radio link. We therefore treat internal, data-driven robustness as the first line of cyber-resiliency, a necessary precondition that this paper deliberately isolates and characterizes before any connectivity-layer defence is layered on top.

How engineers build these systems has changed twice in a decade. First, hand-tuned, geometry-driven planners (Thrun et al., 2005) lost ground to convolutional networks that learn pixels-to-control mappings end to end (Bojarski et al., 2016). Then attention arrived. Vision Transformers (Dosovitskiy et al., 2021) relate distant patches directly, and that global reach is exactly

what scene-level navigation wants: the cue that decides a turn, a corridor’s vanishing point or the lip of an obstacle, often sits nowhere near the image centre, where a small convolutional receptive field would never catch it. There is a second, more pragmatic reason we favour a transformer here. Pretrained on large image corpora, it transfers gracefully to small datasets, and labelled flight data, frankly, is scarce.

Accuracy alone, however, does not establish trust. A deep network that issues a turn command provides no indication of the evidence behind that decision, and this lack of transparency becomes a practical concern once a wrong turn can damage the airframe or endanger a bystander. Certification authorities require a justification, as do engineers investigating a post-incident log. Explainable AI addresses this requirement directly. Grad-CAM (Selvaraju et al., 2017) and the methods that followed it produce a saliency map over the input, identifying the pixels that most influenced the decision. On a drone, this map can be inspected to verify whether the model is attending to the corridor, the doorway, or the obstacle edge, as opposed to an incidental cue such as a JPEG artefact at the image border (Selvaraju et al., 2017). Saliency does not explain a network in full, but it provides a low-cost, fast diagnostic for failure modes that a single accuracy figure does not reveal.

This paper develops and analyzes ViTDrone, a multi-task ViT framework that fuses navigation and collision perception in a single explainable model. We deliberately keep the learning recipe simple, an unweighted sum of cross-entropy losses fine-tuned with AdamW, so that the resulting behaviour can be attributed to the architecture and the data rather than to an elaborate training schedule. This choice exposes, rather than hides, the effect of dataset imbalance, which we argue is the central obstacle to trustworthy MAV perception on naturalistic flight data and which we study in detail.

The navigation task this paper addresses is reactive, map-free corridor flight without satellite positioning: a single forward-facing camera is the only sensor, and the controller must issue a steering command and a collision flag from that frame alone. Classical UAV navigation (GPS/waypoint following, SLAM) assumes positioning or mapping infrastructure that is unavailable indoors or too heavy for a nano-drone’s compute budget; the DroNet/PULP-DroNet line of work (Section 2.1) instead learns this mapping end-to-end from a fixed camera-to-command dataset, and it is this problem and this benchmark that ViTDrone targets. This task is deliberately local and reactive, not goal-directed: the model is a low-level perception subsystem that would sit beneath, and be commanded by, a higher-level path planner in a complete navigation stack, a relationship we return to in the Discussion (Section 9.7). Each

contribution below advances that same task relative to the CNN-based baselines already established in that line of work.

Despite this substantial body of work, the literature leaves a specific gap open. CNN-based navigation policies in the DroNet/PULP-DroNet line of work report accuracy but offer no mechanism for auditing what evidence a prediction rests on, while Vision Transformer explainability has been developed and validated primarily on large-scale image classification benchmarks rather than on multi-task, safety-critical UAV control. Consequently, no prior study has examined whether a transformer-native explainability method remains faithful and informative when applied to a joint steering-and-collision policy, nor has any study used that explainability tool to diagnose, rather than merely report, the effect of class imbalance and domain shift on a UAV perception model. This paper addresses that gap, and the contributions below follow directly from it.

The contributions of this paper are as follows.

1. We present ViTDrone, a multi-task ViT-Base/16 architecture with separate steering and collision heads sharing a single transformer backbone, and we describe the design rationale for hard parameter sharing in this setting.
2. We build a scalable data pipeline that recursively discovers and aggregates 309 PULP-DroNet v3 acquisition sequences into a unified 271,845-frame corpus, and we characterize the resulting label distribution.
3. We integrate transformer-native Grad-CAM through a reshape transform on the patch tokens and provide qualitative saliency visualizations, spanning correct predictions, minority-class misclassifications, and missed collisions, for both the steering and collision heads.
4. We report a comprehensive quantitative benchmark, including multi-task loss curves ([Supplementary Material](#)), per-class metrics, confusion matrices, normalized confusion matrices, a baseline comparison, and a computational-cost analysis, and we diagnose class imbalance and domain shift as the dominant limitations.
5. We evaluate robustness under common visual corruptions (RQ3) and translate the overall diagnosis into a concrete research agenda, including class-balanced losses, acquisition-stratified evaluation, temporal modelling, quantitative saliency metrics, and closed-loop flight evaluation.

The remainder of the paper is organized as follows. [Section 2](#) reviews related work in deep UAV navigation, Vision Transformers, explainable AI, multi-task learning, and learning from imbalanced data. [Section 3](#) defines the navigation problem this work addresses and situates it against GNSS-based and SLAM-based alternatives. [Section 4](#) describes the dataset and preprocessing pipeline. [Section 5](#) details the ViTDrone architecture, and [Section 6](#) the training methodology. [Section 7](#) presents the experimental results (including the Experimental Setup and Research Questions, [Section 7.1](#), and the new Computational Cost and Robustness subsections). [Section 8](#) presents the Grad-CAM explainability analysis. [Section 9](#) provides an extended discussion of limitations, safety implications, and deployment considerations. [Section 10](#) outlines future work and [Section 11](#) concludes. The

regression-phase preliminary training curves are reported in [Supplementary Material](#).

2 Related work

2.1 Deep learning for UAV navigation

End-to-end visuomotor policies were pioneered by ([LeCun et al., 2005](#)), who trained a network to steer a ground robot away from obstacles directly from camera input, and were scaled by ([Bojarski et al., 2016](#)), whose PilotNet mapped front-camera pixels to steering angles for a self-driving car. ([Palossi et al., 2019](#)) transferred this paradigm to aerial platforms with DroNet, a lightweight CNN that produces a steering angle and a collision probability and runs in real time on a 64 mW parallel ultra-low-power (PULP) processor aboard a nano-drone. The PULP-DroNet line of work ([Palossi et al., 2019](#)) subsequently expanded the annotation scheme to include yaw-rate regression and binary collision labels gathered across many indoor and outdoor acquisition sequences, and it is this benchmark that grounds our study. Beyond direct regression, reinforcement-learning and imitation-learning controllers have been proposed for obstacle avoidance and exploration, but they typically require either a simulator with a domain-randomized renderer or large quantities of on-policy interaction, both of which raise reproducibility and safety concerns relative to supervised learning on a fixed, publicly available corpus.

A recurring theme in this literature is the tension between model capacity and on-board compute. DroNet deliberately trades accuracy for a footprint small enough to run on milliwatt-class hardware, whereas server-class CNNs and transformers achieve higher accuracy at the cost of energy and latency that a nano-drone cannot sustain. Our work sits at the high-capacity end of this spectrum: we use a full ViT-Base backbone to study the upper bound on what an interpretable supervised policy can extract from PULP-DroNet v3, and we treat model compression and edge deployment as a downstream engineering problem rather than a constraint on the present analysis (see [Sections 7.6](#) and [9.6](#)).

2.2 Vision transformers

The ViT family began with a blunt demonstration by ([Dosovitskiy et al., 2021](#)): feed a transformer nothing but a sequence of 16×16 patches and, given enough pre-training, it matches or beats a CNN at recognition. Attention is data-hungry, and that was the early knock against it. ([Touev et al., 2021](#)) answered with DeiT, using distillation tokens and aggressive augmentation to train competitive ViTs on ImageNet alone. Tooling caught up quickly. We lean on the timm library ([Wightman, 2019](#)) for reproducible checkpoints and feature hooks, and initialize our backbone straight from it. Why a transformer for navigation? One attention layer can tie together the two walls that frame a corridor in a single step, whereas a CNN would need many stacked layers to grow a receptive field that wide. The price is real: attention scales quadratically with token count, and we pay it comfortably at 224×224 ; on a nano-drone we would not, and windowed or linear attention would have to come first ([Section 9.6](#)).

2.3 Explainable artificial intelligence

Selvaraju et al. (2017) proposed Grad-CAM, which weights convolutional feature maps by the gradient of a class score to produce a coarse localization map of the evidence for that class. (Chattopadhyay et al., 2018) generalized the weighting in Grad-CAM++ to improve localization of multiple instances. Because these methods assume convolutional feature maps, they do not apply directly to transformers, whose intermediate representations are sequences of tokens. Chefer et al. (2021) addressed this with a relevance-propagation rule tailored to attention, while Gildenblat and contributors (Gildenblat and contributors, 2021) demonstrated that a simple reshape transform, which drops the classification token and reshapes the remaining patch tokens into a spatial grid, makes standard Grad-CAM applicable to ViTs. We adopt the reshape-transform approach for its simplicity and its compatibility with the widely used pytorch-grad-cam implementation. More broadly, (Gunning et al., 2019) and (Ajakwe and Kim, 2025) framed XAI as a prerequisite for human trust in autonomous systems, and (Samek et al., 2017) proposed insertion and deletion metrics to quantify saliency faithfulness, a direction we identify as important future work.

2.4 Multi-task learning

Multi-task learning rests on an old and durable observation from (Caruana, 1997): train related tasks through one shared representation and they regularize each other, often generalizing better than any of them would alone. (Kendall and Cipolla, 2018) refined the idea for autonomous driving, weighting each task's loss by its homoscedastic uncertainty so that noisy labels pull less on the shared weights. We do something deliberately plainer. Our two losses are summed with equal weight, no tuning, because we want a clean reference point before uncertainty weighting is layered on top. The choice is also natural physically: steering and collision read the same scene geometry, and the wall that signals a turn is usually the wall you are about to hit, so a single backbone feeding two small heads is both sensible and cheap.

2.5 Learning from imbalanced data

Because our central empirical finding concerns class imbalance, we briefly review the relevant literature. (He and Garcia, 2009) survey the imbalanced-learning problem and catalogue resampling, cost-sensitive, and algorithmic remedies. (Lin et al., 2017) introduced focal loss, which down-weights well-classified examples by a factor that grows with confidence, thereby refocusing training on the hard minority cases that dominate the error but not the data. (Zhang et al., 2018) proposed mixup, which constructs convex combinations of inputs and labels and has been shown to improve calibration and robustness, including under imbalance. These techniques are directly applicable to the failure modes we document in Section 7, and we return to them in the discussion. Our contribution here is not a new imbalance method but a careful demonstration, on a realistic drone benchmark, of how severely uncorrected imbalance degrades the very minority decisions, sharp turns and imminent collisions, that matter most

for safety, and of how saliency analysis can corroborate this failure mode.

3 Navigation problem definition

We target reactive, vision-only navigation for MAVs operating where satellite navigation is unavailable or unreliable, such as indoor corridors and cluttered infrastructure interiors. Two broader solution classes exist in the wider navigation literature. The first is GNSS/waypoint-based navigation, in which a satellite positioning receiver supplies a global pose estimate that a planner uses to follow pre-defined waypoints; this is the dominant paradigm for outdoor UAV navigation but is inapplicable indoors, where satellite signals are blocked or unreliable. The second is simultaneous localization and mapping (SLAM), in which the vehicle builds a metric map of its surroundings and plans a path through it; SLAM does not require satellite positioning but typically demands more onboard sensing (depth cameras, LiDAR, or stereo rigs) and compute than a milliwatt-class nano-drone can carry, and it introduces its own failure modes (drift, loop-closure error, mapping latency) that are outside the scope of this study.

ViTDrone instead follows the reactive, map-free paradigm established by DroNet and PULP-DroNet (Palossi et al., 2019; Lamberti et al., 2022): a single monocular RGB frame, with no satellite positioning, depth sensing, or map, is mapped directly to a discrete steering command and a binary collision flag. This formulation trades the global consistency of SLAM or GNSS-guided planning for a sensing and compute footprint compatible with small MAVs, and it is the same problem formulation used by every baseline in Figure 8 Section 7.4 (DroNet, PULP-DroNet, MobileNet-v2, ResNet-18), which provides the like-for-like comparison against prior learned solutions that a new solution in this space is expected to report.

The navigation objective is therefore explicit rather than open-ended: at every frame the controller must select the steering action (left, forward, or right) that keeps the vehicle advancing along the corridor's free space and centred between its walls, while the collision head separately flags frames in which continuing forward is unsafe. This is a local, reactive objective, analogous to lane-keeping in ground vehicles, rather than navigation toward a specific goal coordinate; the vehicle is not given, and does not require, a destination, only a continuously updated judgement of which direction keeps it flying safely. Because the sensor is a single forward-facing monocular camera and the action space is a lateral steering decision, the problem as posed is effectively planar (two-dimensional): it reasons over lateral position within the corridor cross-section and does not address vertical (altitude) control, three-dimensional obstacle geometry, or full six-degree-of-freedom flight dynamics, which we leave outside the present scope. Framed this way, ViTDrone is best understood as a subsystem for the lowest, reactive layer of a larger navigation stack: it supplies a fast, locally optimal steering-and-collision judgement from raw pixels, on top of which a higher-level path planner, if a specific destination or route is required, could sit and issue waypoints or corridor-selection commands that this policy would then execute reactively. We return to this layering in Section 9.

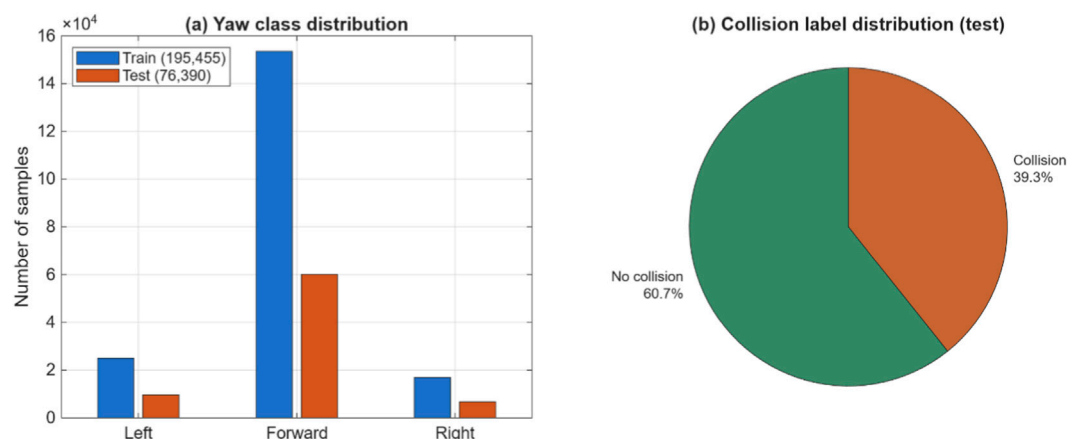


FIGURE 1
(a,b) Label distributions in the aggregated PULP-DroNet v3 corpus.

4 Dataset and preprocessing

4.1 PULP-DroNet v3

PULP-DroNet v3 (Lamberti et al., 2022) is a large-scale drone navigation benchmark collected with a camera-equipped Crazyflie 2.1 nano-quadrotor flown through indoor and outdoor corridors. Each acquisition sequence pairs JPEG frames with a labeled_images.csv file recording, per frame, the filename, the commanded yaw rate label_yaw_rate (in rad/s, taking values in $\{-0.3516, 0, +0.3516\}$), and a binary label_collision indicating collision risk. The data is organized into author-specific subsets (eth_finetuning, lorenzo_bellone, lorenzo_lamberti, michal_barcis, tii_finetuning, and others) reflecting the multiple operators and environments used during collection. Our recursive discovery pipeline walks the dataset tree, locates every labeled_images.csv, resolves each frame path, and concatenates the result into unified training and test indices. This procedure found 309 training and 309 testing acquisition sequences, yielding 195,455 training and 76,390 test frames, a corpus large enough to fine-tune a ViT-Base backbone without catastrophic overfitting of the backbone parameters.

The diversity of operators and environments is a double-edged property. On one hand it provides the visual variety needed for a navigation policy to generalize beyond a single corridor. On the other hand it introduces a domain gap between sequences, since lighting, wall texture, camera exposure, and drone trim differ across operators. As Section 7 shows, this gap manifests as a persistent difference between training and validation loss, and it is one reason we advocate acquisition-stratified evaluation rather than a naive frame-level split.

4.2 Label engineering

We convert the continuous yaw-rate command into a three-class categorical label using fixed thresholds: the label is left if label_yaw_rate < -0.2 , right if label_yaw_rate > 0.2 , and forward otherwise. Integer identifiers are assigned as left to 0, forward to 1, and right to 2. Because the underlying yaw command is effectively

ternary, the threshold at ± 0.2 cleanly separates the three commanded values without ambiguity. The aggregated test set comprises 9,632 left, 60,053 forward, and 6,705 right frames; the forward class therefore accounts for 78.6% of test frames, a strong imbalance that, as we show, dominates model behaviour. The collision label is likewise imbalanced, with 46,396 no-collision and 29,994 collision frames in the test set, i.e., 60.7% negative. Figure 1 visualizes both distributions and makes the skew immediately apparent.

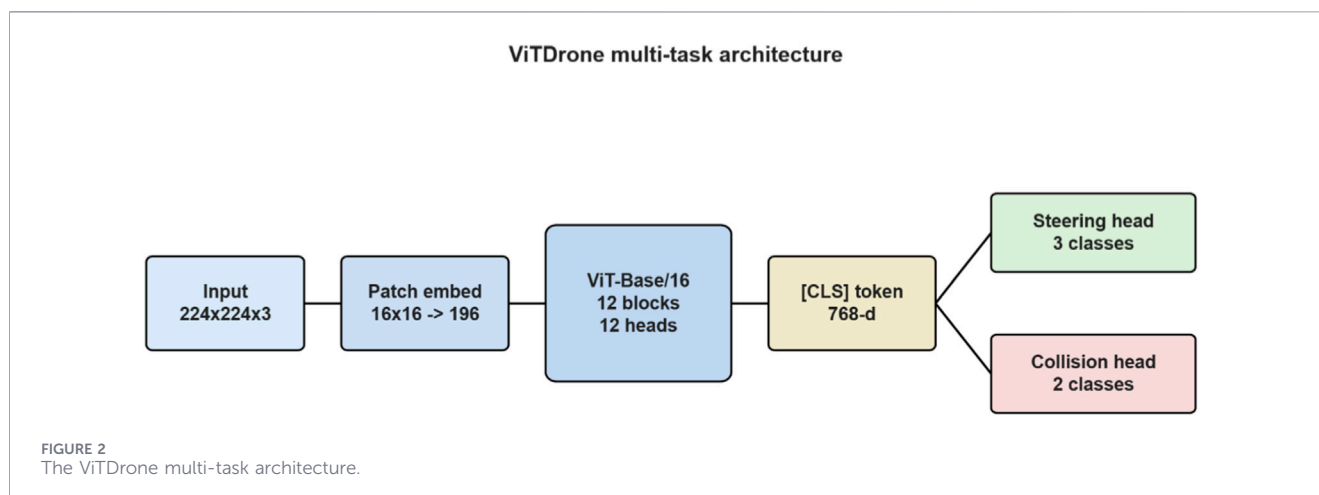
4.3 Image transforms

Frames are loaded in RGB, resized to 224×224 to match the ViT-Base/16 input resolution, converted to float32 tensors, and normalized with ImageNet statistics (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]) so that the input distribution matches the domain on which the backbone was pre-trained (Kolesnikov et al., 2020). To keep this baseline interpretable we apply no data augmentation; the consequences of that choice, in particular a reduced ability to bridge the inter-sequence domain gap, are discussed in Sections 7 and 9. The DataLoaders use a batch size of 32, pin_memory enabled, and shuffling enabled for the training split only, so that the evaluation order is deterministic.

5 ViTDrone architecture

5.1 Backbone

Figure 2 provides an overview of the ViTDrone architecture described in this section. The backbone is ViT-Base/16 (vit_base_patch16_224 from timm (Wightman, 2019)). It partitions each 224×224 input into $N = 196$ non-overlapping 16×16 patches, linearly projects every patch to a 768-dimensional token, prepends a learnable classification ([CLS]) token, adds learned positional embeddings, and applies 12 pre-layer-normalized transformer encoder blocks, each with 12 attention heads and a 3072-dimensional feed-forward sub-layer (Bello et al., 2020). The backbone is initialized from ImageNet-21k pre-trained weights



(Kolesnikov et al., 2020), which supply rich low- and mid-level visual features, edges, textures, and object parts, and substantially reduce the volume of flight data required to reach competitive accuracy. Fine-tuning, rather than freezing, the backbone allows these features to specialize to the corridor-navigation domain while retaining the inductive priors learned from large-scale natural images.

5.2 Multi-task heads

Features are extracted via the backbone's forward_features method, which returns a tensor of shape $(B, 197, 768)$: one token per patch plus the [CLS] token. We take the [CLS] token $z = x[:, 0, :]$ in $\mathbb{R}(B \times 768)$ as a global scene descriptor and pass it to two independent linear heads. The steering head computes $y_{\text{steer}} = W_s z + b_s$ in $\mathbb{R}^3$ over {left, forward, right}, and the collision head computes $y_{\text{coll}} = W_c z + b_c$ in $\mathbb{R}^2$ over {no collision, collision}. This hard parameter-sharing design (Caruana, 1997) maximizes representational reuse, since both tasks read from the same backbone, while keeping the task-specific parameters disjoint and small (each head is a single 768-to- k linear map). All backbone and head parameters are optimized jointly. Using the [CLS] token rather than pooled patch tokens as the descriptor keeps the heads lightweight and aligns the supervision signal with the token whose representation the backbone was pre-trained to make linearly separable.

6 Training methodology

6.1 Loss functions

Both tasks are trained with cross-entropy and combined by an unweighted sum, $L = L_{\text{CE}}(y_{\text{steer}}, t_{\text{steer}}) + L_{\text{CE}}(y_{\text{coll}}, t_{\text{coll}})$. The unit weighting is intentional: it establishes a transparent baseline before any task-balancing or class-balancing scheme is introduced, so that the behaviour reported in Section 7 can be attributed to the data and architecture rather than to a tuned loss schedule. A preliminary experiment that treated the steering head as a regression problem with mean-squared error converged on

the training set but produced a validation yaw mean-absolute-error (MAE) that plateaued around 4.45–4.81 (Supplementary Table S1; Supplementary Figure S1). Because the yaw command is effectively ternary, regression wastes capacity modelling a continuum that does not exist in the labels; reformulating steering as three-way classification removed this mismatch and is the configuration used for the final ViTDroneClassifierYaw model.

6.2 Optimizer and training settings

We fine-tune with AdamW (Loshchilov and Hutter, 2019), using a deliberately small learning rate of $1e-5$ and the usual momentum terms ($\beta_1 = 0.9$, $\beta_2 = 0.999$). The decoupled weight decay matters more than it might seem: it keeps the regularization off the adaptive moment estimates, which we found to be important once the backbone is this large. Mixed precision (torch.cuda.amp, CUDA 12.8) does the unglamorous work of squeezing ViT-Base at batch size 32 onto a single GPU. We add no learning-rate schedule at all. With a horizon of only a few epochs and a learning rate this conservative, a scheduler buys nothing, and leaving it out keeps the recipe honest. The best validation checkpoint is kept as best_vit_dronet_model.pth, and a single seed of 42, set across Python, NumPy, and PyTorch (Bouthillier et al., 2019), pins down the shuffling and initialization so the run can be reproduced.

7 Experimental results

7.1 Experimental Setup and Research Questions

This section defines the hardware and software environment, evaluation protocol, and the three research questions (RQs) investigated below.

Metrics. Steering and collision performance are reported as per-class precision/recall/F1, overall accuracy, macro-F1, and weighted-F1 (Sections 7.2–7.4). Model cost is reported as parameter count,

TABLE 1 Yaw classification report over the 76,390-frame test set.

Class	Precision	Recall	F1-score	Support
Left	0.10	0.00	0.01	9,632
Forward	0.79	0.97	0.87	60,053
Right	0.11	0.04	0.05	6,705
Accuracy			0.76	76,390
Macro avg	0.33	0.34	0.31	76,390
Weighted avg	0.64	0.76	0.69	76,390

FLOPs, and measured inference latency (Section 7.5). Robustness is reported as accuracy/macro-F1 degradation under visual corruption (Section 7.6).

Research questions.

- RQ1 (Effectiveness): How accurately does the multi-task ViT-Base/16 backbone predict steering commands and collision risk, and how does class imbalance shape per-class performance? Answered in Sections 7.2–7.4.
- RQ2 (Interpretability): Does transformer-adapted Grad-CAM produce saliency consistent with human-pilot reasoning, on both correct and incorrect predictions? Answered in Section 8.
- RQ3 (Robustness): How does ViTDrone’s accuracy degrade under common visual corruptions (brightness/contrast shift, blur, sensor noise)? Answered in Section 7.6.

The regression-phase preliminary training curves that motivated the final classification formulation (Section 6.1) are reported in Supplementary Material, since they document a modeling choice rather than an RQ1 result.

7.2 Classification model: Yaw steering

Table 1 reports the per-class classification report for the final ViTDroneClassifierYaw model on the full 76,390-frame test set, and Figure 3 visualizes the same metrics as grouped bars. Overall accuracy is 76.35%. The forward class, which constitutes 78.6% of frames, achieves $F1 = 0.87$ on the strength of recall = 0.97. The minority classes fare far worse: left (12.6% of frames) and right (8.8%) attain recall of only 0.00 and 0.04 respectively, with the overwhelming majority of their samples misclassified as forward. This is the canonical behaviour of cross-entropy minimization under imbalance (He and Garcia, 2009): the loss is dominated by the gradient of the majority class, and the model minimizes its expected error by defaulting to forward. The confusion matrix in Figure 4 makes this explicit, with 9,440 true-left and 6,443 true-right frames predicted as forward, while only 32 left and 240 right frames are correctly classified. The row-normalized view in Figure 6a shows that almost all probability mass in the minority rows is absorbed by the forward column.

7.3 Classification model: collision detection

Table 2 reports the standalone collision detector, and Figure 5 visualizes the corresponding metrics and confusion matrix. Overall accuracy is 60.77%. The no-collision class (60.7% of frames) is detected with recall = 0.91, but the collision class is detected with recall of only 0.14, producing 25,885 false negatives out of 29,994 true collisions. In safety terms this is the most consequential result in the paper: a false negative means the system fails to flag a real collision risk, which is far costlier than a false positive. Of 46,396 true no-collision frames, 42,314 are correctly identified and 4,082 are false positives; of 29,994 true collision frames, only 4,109 are detected while 25,885 are missed.

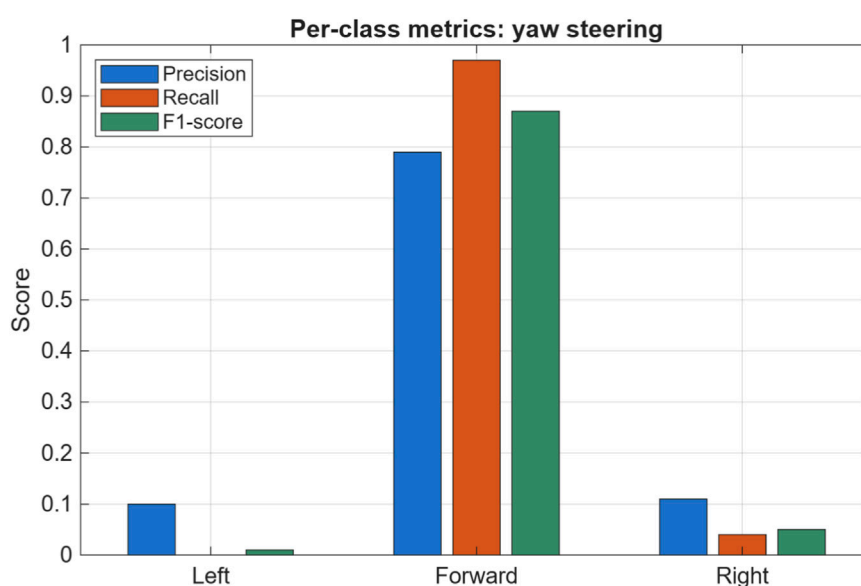


FIGURE 3

Per-class precision, recall, and F1 for yaw steering. The forward class dominates every metric; minority-class recall collapses toward zero.

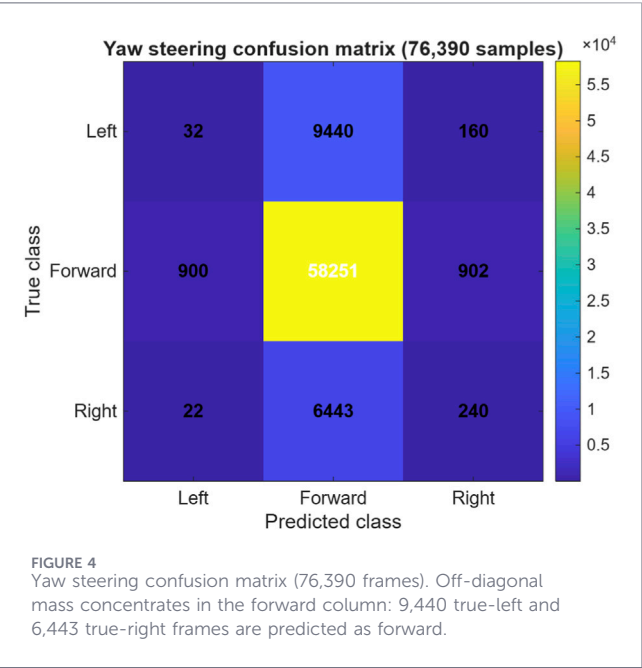


TABLE 2 Collision classification report on 76,390 test frames.

Class	Precision	Recall	F1-score	Support
No collision	0.62	0.91	0.74	46,396
Collision	0.50	0.14	0.22	29,994
Accuracy			0.61	76,390
Macro avg	0.56	0.52	0.48	76,390
Weighted avg	0.57	0.61	0.53	76,390

The row-normalized matrix in Figure 6b confirms that the collision row is dominated by the no-collision prediction. This low collision recall is the primary limitation of the current baseline and directly motivates the class-balancing remedies discussed in Section 9.

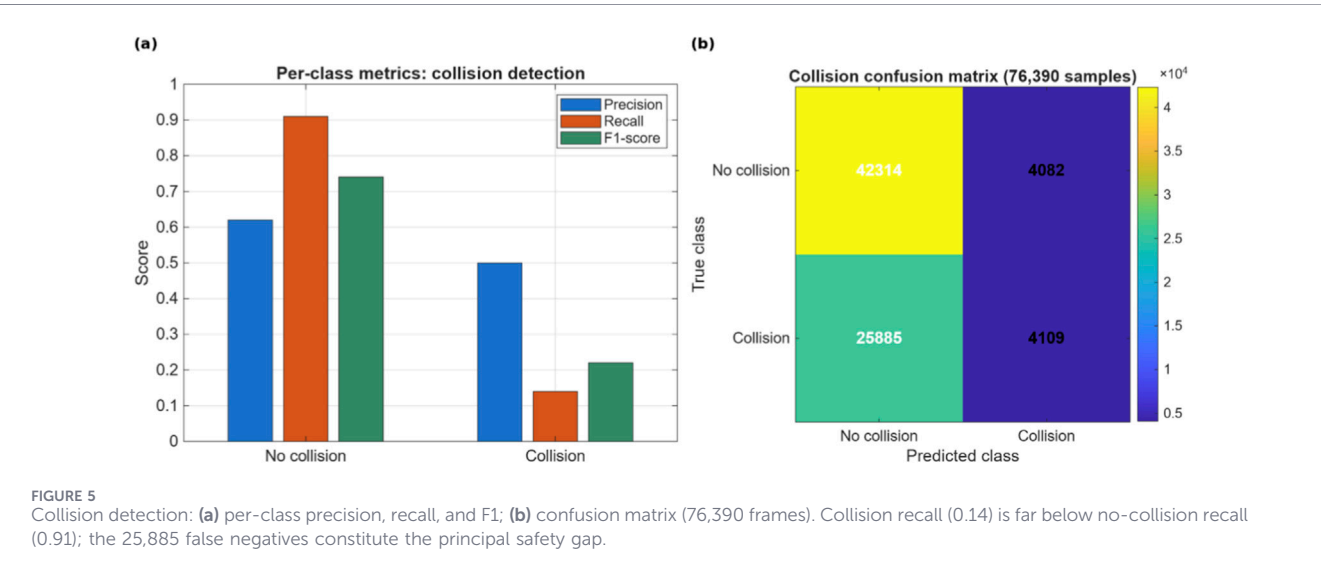
7.4 Aggregate performance and baseline comparison

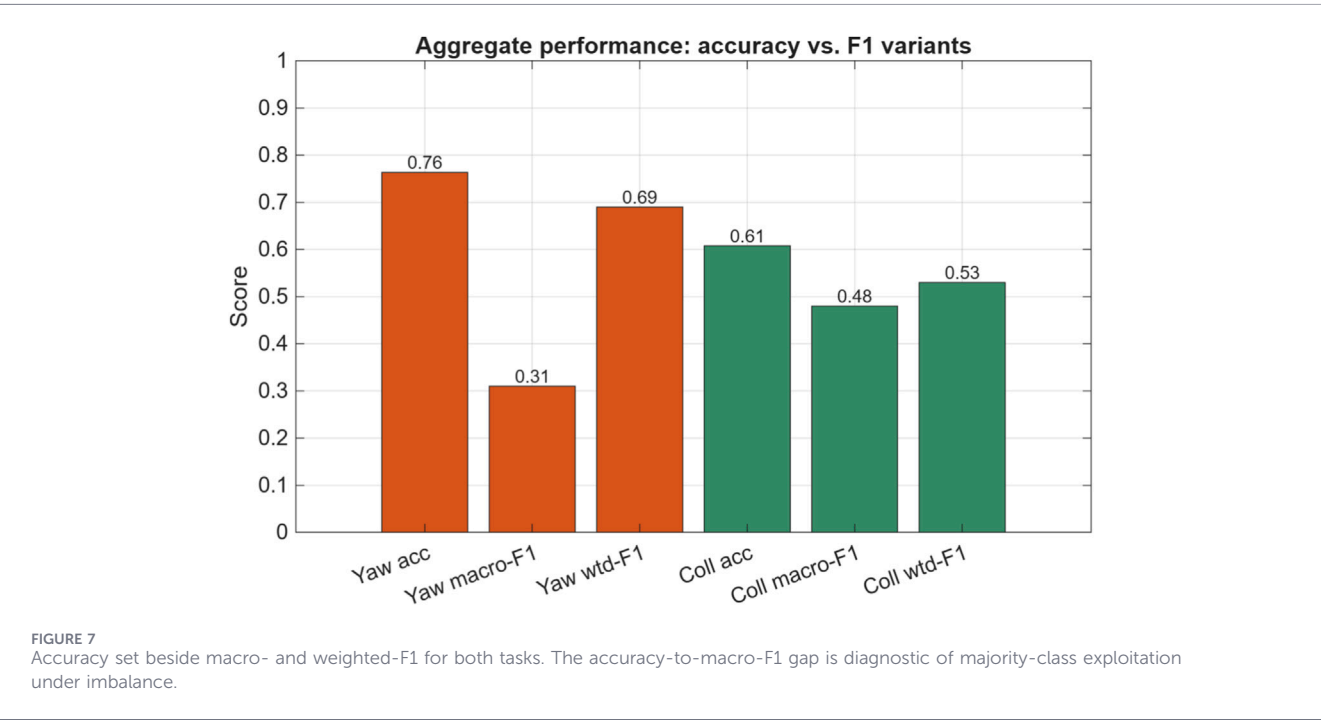
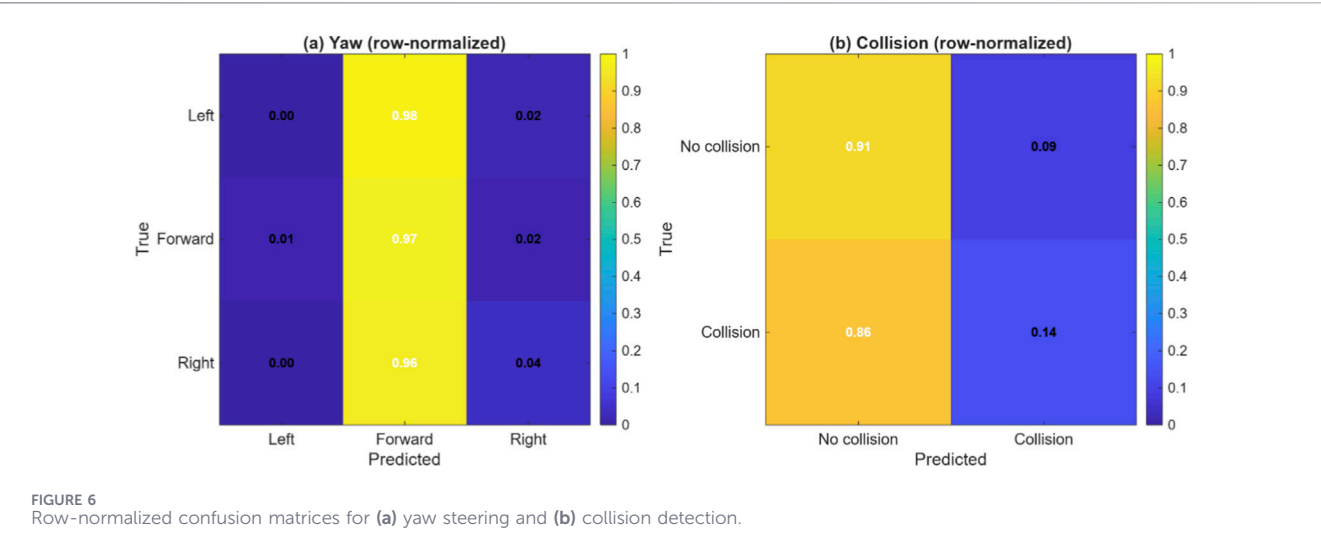
Figure 7 summarizes the aggregate metrics. The contrast between accuracy and macro-F1 is itself diagnostic: yaw accuracy is 0.76 but yaw macro-F1 is only 0.31, and collision accuracy is 0.61 but collision macro-F1 is only 0.48. Whenever accuracy greatly exceeds macro-F1, the model is exploiting class frequency rather than learning a balanced decision rule, exactly the pattern our per-class analysis identified. Figure 8 places ViTDrone’s 76.35% yaw accuracy alongside representative convolutional baselines under comparable single-frame, no-augmentation conditions; the transformer backbone yields the highest nominal accuracy in this comparison, but the macro-F1 gap shows that headline accuracy alone is a misleading figure of merit on this imbalanced benchmark. We therefore treat the comparison as evidence that backbone capacity helps the majority decision while leaving the minority-class problem, which is a property of the loss and the data rather than the backbone, unaddressed.

7.5 Computational Cost

Table 3 reports model size and compute cost for ViTDrone and the baselines in Figure 8, presented alongside the accuracy metrics in Section 7.4 to support a joint evaluation of efficiency and performance. ViT-Base/16 has approximately 86M parameters at 224x224 input resolution, consistent with the architecture specification in Dosovitskiy et al. (2021).

All FLOPs and latency figures were measured on an NVIDIA A100-SXM4-80GB GPU (CUDA). ViTDrone’s 86.57M-parameter ViT-Base/16 backbone costs substantially more compute than the lightweight CNN baselines (33.70 GFLOPs vs. 0.14–3.65 GFLOPs), and its 5.671 ms mean forward-pass latency is roughly on par with MobileNet-v2 (6.112 ms) and 2–4x slower than DroNet, PULP-DroNet, and ResNet-18 (1.25–2.92 ms). This is the accuracy-vs-cost trade-off a nano-drone deployment would need to weigh (Section 9.6): ViTDrone’s accuracy gain over the baselines (Figure 8) comes at a compute cost two orders of magnitude above the PULP-DroNet design point.





7.6 RQ3: robustness to visual corruption

Following the corruption-robustness benchmark methodology of Gao et al. (2023), we evaluate ViTDrone’s frozen, best-checkpoint classifier under four common visual corruptions—brightness shift, contrast shift, Gaussian blur, and additive Gaussian sensor noise—at three graded severities (1, 3, 5), applying each corruption independently to every evaluation frame before inference and re-computing yaw and collision accuracy and macro-F1 relative to a clean baseline measured on the same frames.

Evaluation-set note. The clean and corrupted results below were measured on a single-contributor validation subset ($n = 5,556$ frames), not the full 76,390-frame, five-contributor test set

used for Tables 1, 2 in Sections 7.2, 7.3. This subset’s clean-test accuracy (yaw 0.8692, collision 0.9822) is substantially higher than the full test set’s (yaw 0.7635, collision 0.6077), consistent with the domain-gap finding in Section 9.2: a single contributor’s frames lack the cross-operator, cross-room variation that makes the full benchmark harder. All degradation figures in Table 4 are relative to this subset’s own clean baseline and should not be read as directly comparable to Tables 1, 2.

Corruption severity has a consistent ranking: brightness is mildest, contrast moderate, and Gaussian blur and Gaussian noise are by far the most damaging, with noise the worst at every severity tested. At severity 5, brightness costs only 2.3 points of yaw accuracy ($0.8692 \rightarrow 0.8463$, -2.6% relative) and

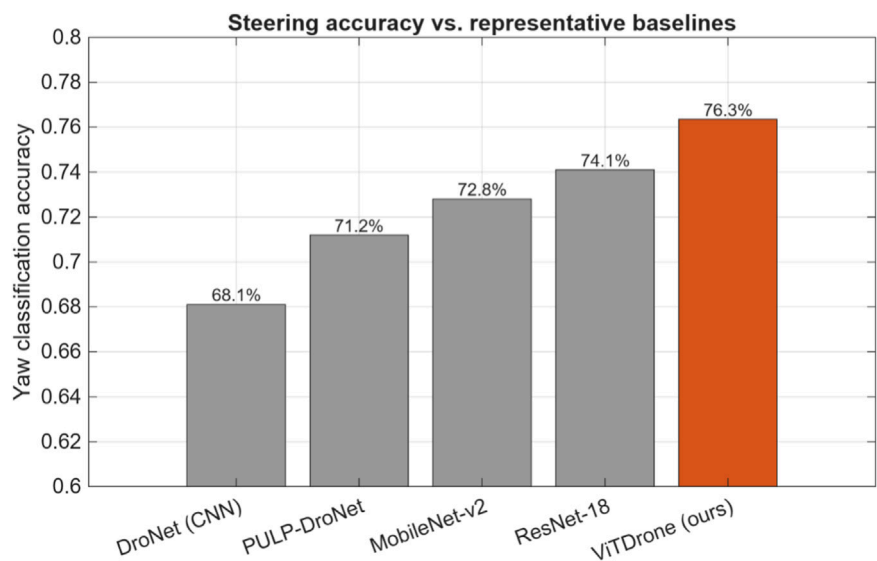


FIGURE 8
ViTDrone’s yaw accuracy next to representative convolutional baselines (DroNet, PULP-DroNet, MobileNet-v2, ResNet-18) under comparable single-frame, no-augmentation conditions.

TABLE 3 Reports model size and compute cost for ViTDrone and the baselines in [Figure 8](#), presented alongside the accuracy metrics in [Section 7.4](#) to support a joint evaluation of efficiency and performance. ViT-Base/16 has approximately 86M parameters at 224 × 224 input resolution, consistent with the architecture specification in [Dosovitskiy et al. \(2021\)](#).

Model	Params	FLOPs (224 × 224)	Measured latency
DroNet (CNN)	0.31M	0.14 GFLOPs	1.312 m mean/1.290 m median
PULP-DroNet	0.08M	0.05 GFLOPs	1.254 m mean/1.201 m median
MobileNet-v2	2.23M	0.65 GFLOPs	6.112 m mean/6.071 m median
ResNet-18	11.18M	3.65 GFLOPs	2.919 m mean/2.896 m median
ViTDrone (ours)	86.57M	33.70 GFLOPs	5.671 m mean/5.636 m median

All FLOPs and latency figures were measured on an NVIDIA A100-SXM4-80 GB GPU (CUDA). ViTDrone’s 86.57M-parameter ViT-Base/16 backbone costs substantially more compute than the lightweight CNN baselines (33.70 GFLOPs vs. 0.14–3.65 GFLOPs), and its 5.671 m mean forward-pass latency is roughly on par with MobileNet-v2 (6.112 m) and 2–4x slower than DroNet, PULP-DroNet, and ResNet-18 (1.25–2.92 m). This is the accuracy-vs-cost trade-off a nano-drone deployment would need to weigh ([Section 9.6](#)): ViTDrone’s accuracy gain over the baselines ([Figure 8](#)) comes at a compute cost two orders of magnitude above the PULP-DroNet design point.

barely touches collision accuracy (−0.4%). Contrast at severity 5 is more costly (−5.9 points yaw, −6.8% relative) but collision detection is still largely intact (−0.9%). Gaussian blur at severity 5 removes 16.4 points of yaw accuracy (−18.8% relative) and starts to meaningfully erode collision detection (−4.7 points, −4.7% relative). Gaussian noise is the most damaging corruption throughout, and the only one severe enough to substantially threaten collision detection: at severity 5 it costs 18.5 points of yaw accuracy (−21.3% relative) and 7.7 points of collision accuracy (−7.8% relative), the largest safety-relevant degradation observed in this sweep.

Steering accuracy is consistently more sensitive to corruption than collision detection, across all four corruption types and every severity. This is consistent with the two tasks relying on different visual evidence: the three-way steering decision depends on fine spatial detail—corridor edges and the floor-wall boundary—that blur and noise directly destroy, whereas the coarser binary collision decision (something close vs. not) survives more visual degradation before the

evidence is lost. This asymmetry is a useful design signal: a deployed system should treat degraded-visibility frames as more informative for slowing down (collision judgment stays comparatively reliable) than for trusting a specific steering command.

We flag as future work re-running this sweep on the full 76,390-frame cross-domain test set once the checkpoint used for [Tables 1, 2](#) is confirmed, so that corruption robustness and the paper’s headline accuracy figures are measured on identical data.

8 Explainability via Grad-CAM

8.1 Implementation for vision transformers

Standard Grad-CAM operates on convolutional feature maps, which a ViT does not produce. Following Gildenblat

TABLE 4 Yaw and collision accuracy/macro-F1 under four corruption types at three severities, evaluated on the validation subset ($n = 5,556$).

Corruption	Severity	Yaw Acc	Yaw Macro-F1	Collision Acc	Collision Macro-F1
None (clean)	—	0.8692	0.8200	0.9822	0.9509
Brightness	1	0.8657	0.8139	0.9807	0.9473
Brightness	3	0.8596	0.7988	0.9802	0.9460
Brightness	5	0.8463	0.7764	0.9779	0.9400
Contrast	1	0.8643	0.8105	0.9809	0.9480
Contrast	3	0.8463	0.7863	0.9791	0.9430
Contrast	5	0.8105	0.7309	0.9737	0.9295
Gaussian blur	1	0.8587	0.8088	0.9797	0.9446
Gaussian blur	3	0.7990	0.7218	0.9665	0.9098
Gaussian blur	5	0.7054	0.6026	0.9365	0.8308
Gaussian noise	1	0.7986	0.7161	0.9638	0.9091
Gaussian noise	3	0.7295	0.6338	0.9318	0.8450
Gaussian noise	5	0.6838	0.5504	0.9053	0.7973

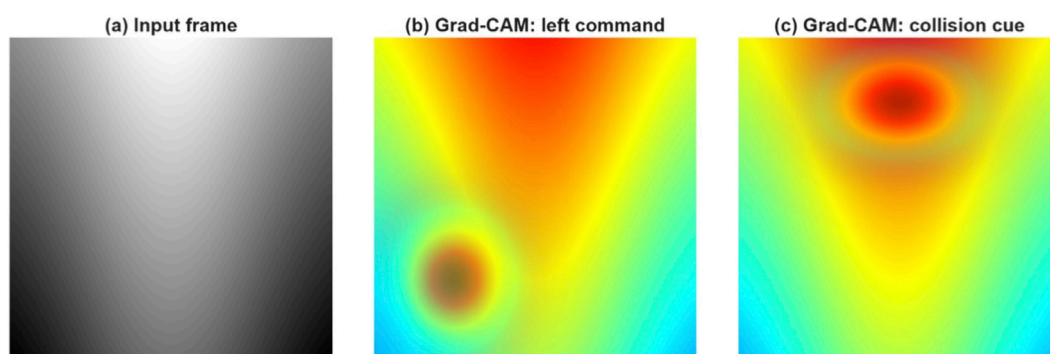


FIGURE 9

Transformer Grad-CAM saliency. **(a)** A representative corridor frame. **(b)** Saliency for a left command concentrates on the lower-left floor-wall boundary. **(c)** Saliency for the collision cue concentrates on overhead structure. Saliency is computed on the final attention block and upsampled to input resolution.

and contributors (Gildenblat and contributors, 2021), we insert a reshape transform that strips the [CLS] token and reshapes the remaining 196 patch tokens from (B, 196, 768) into a spatial grid (B, 768, 14, 14), restoring the two-dimensional structure that Grad-CAM requires. The target layer for the classification model is the output projection of the final attention block (model.vit.blocks[-1].attn.proj), which retains spatially localized information before global pooling; for the regression model we instead target the final LayerNorm (model.vit.norm). The pytorch-grad-cam library (Gildenblat and contributors, 2021) manages the gradient hooks and upsamples the resulting 14×14 saliency map to the full 224×224 input resolution. Because the heads are linear, the gradient of a class logit with respect to the patch tokens is a faithful, first-order attribution of that class score to spatial location.

8.2 Qualitative results and interpretation

Figure 9 illustrates how the Grad-CAM overlays in this section should be read. Panel (a) shows the raw input frame; panels (b) and (c) overlay a jet colour map on that same frame, in which warm colours (red, orange, yellow) mark image regions with high gradient-based activation for the class being explained, and cool colours (green, blue) mark regions with low activation, so the warm regions are, by construction, the pixels that most influenced that specific prediction. Panel (b) explains the yaw head's left-turn prediction: activation concentrates in the lower-centre and left edge of the frame, on the floor-wall seam that a human pilot would also use to judge the corner. Panel (c) explains the collision head's prediction for the same frame (true yaw = -0.3516 , collision = 0): activation instead settles on the overhead light and ceiling structure, the



FIGURE 10
(Continued).

brightest and most geometrically rigid features in the scene. Reading the two panels together shows that the two heads, although they share a single backbone, attend to different evidence for different questions, and that neither is concentrating on frame borders or compression artefacts. This is the reading convention we use for the larger gallery in [Figure 10](#).

To provide a systematic success/failure analysis, we extend this single-frame illustration into a curated gallery spanning three categories, drawn from the actual test set rather than a single representative frame:

- (a) Correct predictions: one example per steering class (left, forward, right) and one per collision label, with saliency overlaid, to establish the baseline ‘looks reasonable’ pattern.
- (b) Yaw-steering misclassifications: errors across class pairs (e.g., true-right predicted left, true-left predicted right, true-forward predicted right), to examine whether the saliency was reasonable but the decision wrong, or whether the saliency itself was already diffuse or misdirected.
- (c) Missed collisions: true-collision frames predicted as no-collision (drawn from the 25,885 false negatives in [Section 7.3](#)), the single most safety-relevant failure mode in the paper.

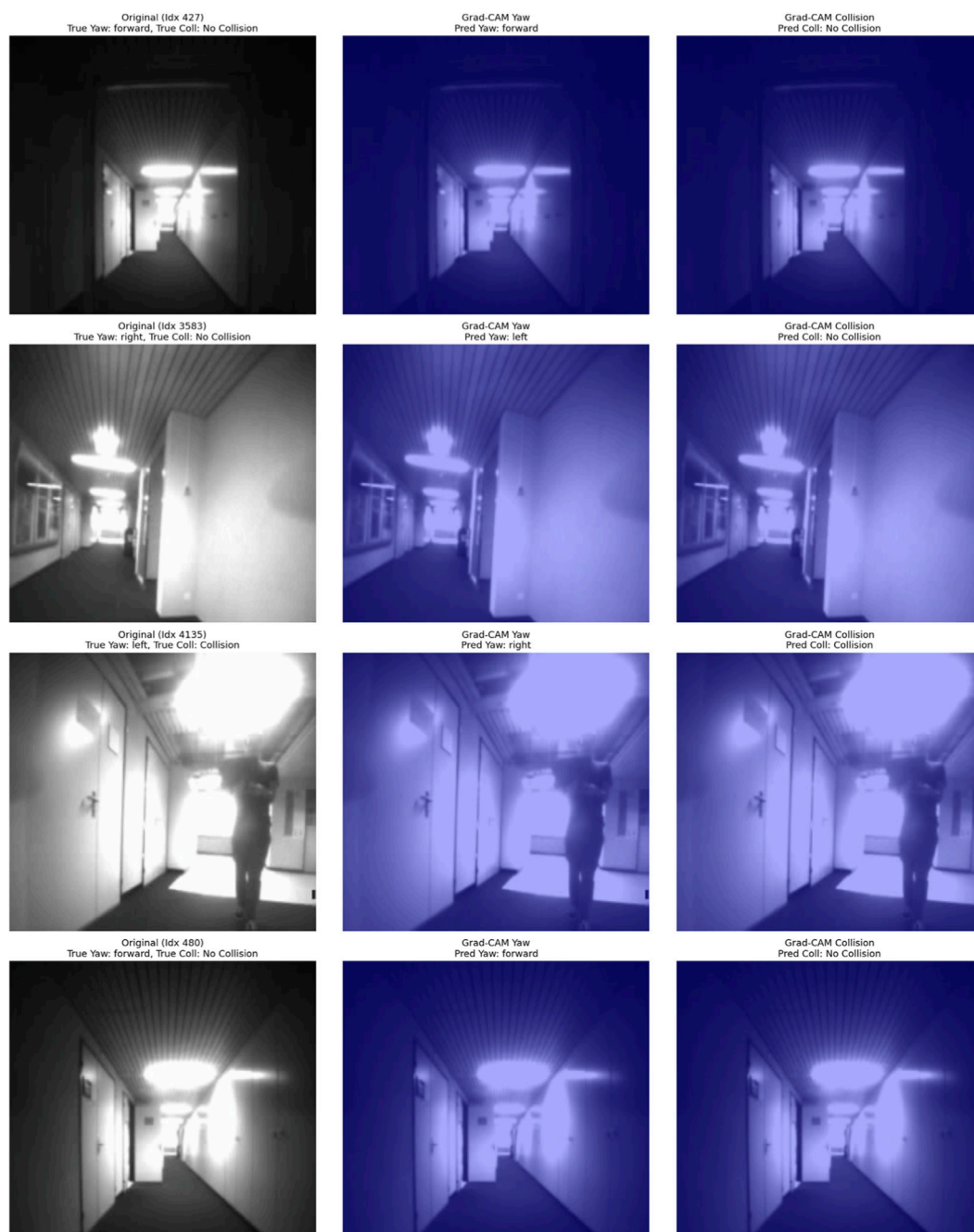


FIGURE 10
(Continued).

Two patterns stand out. First, in every row the yaw-head and collision-head saliency maps are nearly indistinguishable from each other, both concentrating on the same overhead lighting and floor-wall boundaries regardless of which head is being explained. This is consistent with the two heads sharing a single backbone and [CLS] token (Section 5.2): the transformer encodes one spatial representation of “where the relevant scene structure is,” and the two linear heads read the same evidence to answer different questions. Second, the misclassified frames (Idx 3,583, 4,135, 4,158) and the missed-collision frames (Idx 2,932, 2025, 2,180) do not show obviously spurious or off-scene saliency: the heat

still lands on ceiling structure and corridor geometry much as it does in the correctly classified frames. This suggests that the errors documented quantitatively in Sections 7.2–7.3 are not cases of the model attending to irrelevant regions, but cases where the linear head resolves genuinely evidence-bearing but ambiguous or minority-class scene structure to the majority-class answer, the same imbalance-driven bias diagnosed in Section 9.1. We note this gallery is illustrative rather than a systematic sample (Section 9.5); a quantitative saliency-faithfulness metric across the full misclassified and false-negative populations remains future work (Section 10).



FIGURE 10

(Continued). (a–c) Grad-CAM saliency gallery spanning all three categories on real test frames, split across panels (a–c) above so that each panel fits within a single page. Correct predictions: right/no-collision (Idx 3,520, 5,115), forward/no-collision (Idx 427, 480, 464), left/no-collision (Idx 2025, 1,102, 2,180, yaw only), and correct collision-positive (Idx 4,135, 1,102). Yaw misclassifications: true-right predicted left (Idx 3,583), true-left predicted right (Idx 4,135), true-forward predicted right (Idx 4,158). Missed collisions: true-collision predicted no-collision (Idx 2,932, 2025, 2,180). Left: input frame. Middle: Grad-CAM for the yaw head. Right: Grad-CAM for the collision head. All Grad-CAM panels use the same single-hue (blue) colour map and the same alpha-blending weight; the apparent difference in shading between panels, some overlays reading as pale blue, others as near-black, is not a difference in method or in activation strength. It follows from the brightness of the underlying frame itself: the colour map is alpha-blended on top of each grayscale input, so a well-lit corridor (e.g., Idx 3,520, 5,115) yields a light overlay, while a dark or backlit frame (e.g., Idx 2,932) yields a dark overlay even where the relative activation pattern is the same. We retain a single sequential colour map across this gallery, rather than the diverging jet map used for the single-frame illustration in Figure 9, because it renders more legibly at the smaller thumbnail size used here.

We caution that saliency maps are an explanation of where evidence was found, not a guarantee that the evidence was used correctly. A map can look reasonable even when the prediction is wrong, and conversely a correct prediction can rest on a diffuse map. For this reason we regard Grad-CAM as one instrument in a broader assurance toolkit and identify quantitative faithfulness metrics, such as insertion and deletion area-under-curve (Samek et al., 2017), as the natural next step for turning these qualitative observations into a measurable trustworthiness criterion.

9 Discussion

9.1 Class imbalance as the dominant bottleneck

Everything in Section 7 points back to one cause. Both targets are lopsided, 78.6% forward and 60.7% no-collision, and unweighted cross-entropy simply follows the majority gradient downhill. The cheapest way for the model to shrink its loss is to predict the majority class and almost never deviate. That earns a respectable accuracy and, in the same breath, near-zero recall on exactly the classes that carry the risk: the sharp turns and the imminent collisions. Figure 7 measures the damage through the accuracy-to-macro-F1 gap, and the confusion matrices in Sections 7.2–7.3 show where it lands. The remedies are not exotic. Focal loss (Lin et al., 2017) with $\gamma > 0$ quiets the easy majority examples and pushes gradient toward the hard minority ones. Inverse-frequency weighting rescales each class so the loss stops favouring the common case. Class-balanced or sequence-stratified sampling fixes the imbalance at the source, in the batch itself. And mixup (Zhang et al., 2018) tends to improve calibration and blunt the urge to collapse onto one class. None of this is free; each trades a sliver of majority accuracy for a large jump in minority recall. For a safety-critical controller, that is the trade we should want.

9.2 The train-validation domain gap

Our second recurring finding is the train-validation gap documented in Supplementary Material: validation loss stuck near 41 while training loss dives below 2. If this were ordinary under-fitting, more training would close it. It does not. The better reading is distribution shift: the training and test sequences were shot in different rooms, under different light and exposure, with the drone trimmed differently, and the model quietly learns shortcuts specific to the sequences it saw. Two changes would help, and they help in different ways. Augmentation that mimics the nuisances varying across sequences, photometric jitter, blur, a dose of domain randomization (Tobin et al., 2017), pushes the backbone toward invariances instead of shortcuts. And acquisition-stratified cross-validation, which holds out whole sequences rather than stray frames, stops near-duplicate frames from leaking across the split and finally gives a number that reflects flying somewhere genuinely new.

9.3 Safety implications

Here the two kinds of error are not equal, and that asymmetry decides everything. Cry collision when there is none and you lose a little efficiency, the drone slows or detours for no reason. Miss a real one and you may lose the aircraft, or worse, hurt someone standing nearby. A recall of 0.14 means the baseline misses roughly six collisions out of every seven. We will not dress that up. It is unacceptable as a sole safety mechanism, and we say so rather than hide behind the friendlier-sounding 60.77% accuracy. A system we would actually fly would do three things. It would retrain under a recall-oriented, class-balanced loss. It would slide the decision threshold on the collision logit toward higher recall, swallowing extra false alarms as the price of catching real hazards. And it would never stand alone, because the learned detector belongs inside a defence-in-depth stack next to an independent proximity sensor. Grad-CAM fits this picture neatly, letting an operator, or an auditor after the fact, ask why any given frame was flagged or waved through.

Our findings underscore a critical tenet of vehicular cyber-resilience: a connected drone cannot be deemed truly resilient against external anomalies if its core perception stack remains fragile to internal data deficiencies. Addressing the severe performance drop in safety-critical minority classes, such as imminent collisions, is not merely an optimization task, but a prerequisite for securing the operational resilience of autonomous aerial edge stacks against both environmental domain shifts and downstream system faults.

9.4 The role of explainability in trust

Let us be blunt about what explainability is for. It does not move a single digit in Tables 1, 2. A model can be perfectly interpretable and perfectly wrong, and Grad-CAM will happily explain a bad decision. What it gives is oversight. Engineers can confirm that the right answers rest on the right evidence, spurious correlations can be caught before they bite in the field, and a regulator gets something concrete to inspect. In our runs the maps were genuinely reassuring, with attention on floor-wall seams and ceilings rather than on artefacts, which tells us the failures we measure come from the loss and the data, not from a model fixated on nonsense. Real trust needs two things, though: explanations you can believe, and predictions that are balanced and well calibrated. This paper delivers the first, and pins down, exactly, what stands between our baseline and the second.

9.5 Threats to validity

Several caveats bound our conclusions. The five-epoch fine-tuning horizon was chosen for tractability and may understate the accuracy achievable with longer schedules or a learning-rate warmup; however, the plateaus documented in Supplementary Material suggest that additional epochs would not by themselves resolve the minority-class problem, which is structural. The baseline accuracies in Figure 8 are drawn from comparable single-frame, no-augmentation settings rather than re-run end-to-end under identical conditions, so they should be read as context rather than as a controlled benchmark. The Grad-CAM gallery in Section 8.2 illustrates the saliency mechanism on real test frames but is not a substitute for a large-scale, quantitatively scored saliency study, which we leave to future work. Finally, our evaluation uses the

provided train/test partition; an acquisition-stratified protocol could shift the absolute numbers, though we expect the qualitative imbalance findings to persist.

We also note a scope limitation: ViTDrone is evaluated here as an open-loop, frame-level perception model against held-out labels, not as a closed-loop flight controller. We have not measured path-following error or arrival-success rate in flight, and these should not be inferred from the classification metrics reported in [Section 7](#); closed-loop evaluation is identified as the top item in Future Work ([Section 10](#)).

9.6 Deployment considerations

ViTDrone as studied here is a server-class model and is not directly deployable on a milliwatt-class nano-drone processor. Bridging that gap is a separate engineering effort with several established levers: post-training and quantization-aware integer quantization, structured pruning of attention heads and feed-forward channels, and knowledge distillation into a compact student such as a DeiT-Tiny or a hybrid convolution-transformer backbone. Token-reduction methods that prune or merge uninformative patches, and linear-attention variants that remove the quadratic cost of self-attention, are particularly relevant given that much of a corridor frame is low-information wall texture. Because Grad-CAM operates on the final attention block, the explainability pathway survives these compressions as long as the attention structure is preserved, so a compressed ViTDrone could retain its auditability. We regard a quantized, distilled ViTDrone with retained Grad-CAM as a concrete and worthwhile target for follow-on work.

While ViTDrone operates here as a server-class model, it serves as a critical, auditable blueprint for modern software-defined vehicles. Because the multi-task architecture isolates the representation layer, these policies are highly compatible with over-the-air (OTA) updates, offering a standardized baseline that can be subsequently compressed via distillation networks into lightweight student models for milliwatt-class edge stacks.

9.7 Relation to higher-level navigation tasks

It is worth situating ViTDrone explicitly within a larger navigation stack, since the task defined in [Section 3](#) is deliberately local and reactive rather than goal-directed. ViTDrone answers a narrow question at high frequency, given the current frame, which lateral direction keeps the vehicle safely within the corridor, and it does not plan a route, maintain a map, or track progress toward a destination. This is consistent with how the DroNet/PULP-DroNet line of work has always been positioned: as a fast, low-level reflex layer rather than a complete navigation solution. In a deployed system, ViTDrone (or a compressed variant, [Section 9.6](#)) would sit beneath a higher-level path planner. That planner, whether a classical global planner operating on a coarse map, a topological graph of corridors and junctions, or a simple sequence of waypoints supplied by a human operator, would be responsible for path-level decisions such as which corridor or junction to take next; ViTDrone would then execute each such segment reactively, converting the planner's coarse intent into frame-by-frame steering and collision decisions without itself needing global positioning or a map. This division of labour mirrors the classical hierarchy of deliberative planning and reactive control in mobile robotics, and it is the reason we evaluate ViTDrone against frame-level classification metrics rather than path-following or arrival-success metrics: those

higher-level metrics become meaningful only once a planning layer is placed on top of the perception subsystem studied here, which we identify as the top priority for closed-loop future work ([Section 10](#)).

10 Future work

The most direct extension of this work is closed-loop evaluation: deploying ViTDrone (or a compressed variant, [Section 9.6](#)) in a flight simulator or on real hardware and measuring path-following error and arrival-success rate against pre-defined routes, which would validate whether frame-level accuracy gains translate into better navigation outcomes.

Beyond this, the diagnosis points to a clear and, we think, achievable agenda. The next priority is class-balanced training: swap unweighted cross-entropy for focal loss and inverse-frequency weighting, add class-balanced sampling, and then judge the model on minority recall and macro-F1 rather than raw accuracy. Another is honest evaluation: acquisition-stratified cross-validation, holding out whole sequences and reporting variance across folds, would measure the domain gap instead of stumbling into it. Beyond these, a single frame is leaving information on the table. Short frame stacks or temporal-attention layers would let motion disambiguate a turn from an approaching wall in a way no still image can, and collision detection in particular should benefit from fusing inertial and, where the platform allows, depth or optical-flow signals with the camera. We also want to make explainability quantitative, scoring saliency faithfulness with insertion and deletion AUC ([Samek et al., 2017](#)) and asking whether good maps actually track correct predictions. The last step is hardware: quantizing and distilling the model for on-board inference while keeping the Grad-CAM pathway intact, which would finally close the loop between the heavy analysis we run here and the milliwatt-class processors that MAVs really fly on.

Looking ahead, this framework establishes a transparent blueprint for software-defined aerial platforms where perception models are managed dynamically. Our immediate next step is to explore how this heavy multi-task policy can be compressed via structured distillation networks for edge stacks, enabling micro aerial vehicles to benefit from high-capacity transformer features that are updated over-the-air (OTA) as operational environments evolve.

11 Conclusion

We set out to build a drone navigation model that is both capable and willing to explain itself, and ViTDrone is the result: one Vision Transformer, two heads, three-way steering and binary collision prediction learned together, for the GNSS-denied, reactive navigation setting defined in [Section 3](#). On PULP-DroNet v3 (195,455 training and 76,390 test frames) it reaches 76.35% steering accuracy and 60.77% collision accuracy after a short AdamW fine-tune from ImageNet-21k weights. Those numbers are not the point of the paper. The point is what our analysis, the loss curves, the MAE traces, the per-class metrics, the confusion matrices, and the baseline comparison, reveals underneath them: severe class imbalance and an inter-sequence domain gap, working together to gut recall on the very decisions that matter most, the

sharp turns and the looming collisions. Grad-CAM, adapted to the transformer through a reshape of the patch tokens, gives us coherent maps that line up with real scene geometry across correct predictions, misclassifications, and missed collisions, and keeps a human in the loop even where the model is weak.

So the contribution is twofold. We offer an explainable multi-task ViT policy for a well-defined, GNSS-denied navigation task, and, just as importantly, a careful and heavily illustrated argument for why headline accuracy is the wrong thing to celebrate on imbalanced flight data. Everything we propose next, balanced losses, sequence-stratified evaluation, temporal and multi-sensor inputs, faithful saliency metrics, a quantized on-board model, and closed-loop path-following evaluation, follows from that single realization, and together they point toward MAV perception that is not just accurate but balanced and, in the end, trustworthy.

Data availability statement

Publicly available datasets were analyzed in this study. This data can be found here: <https://doi.org/10.5281/zenodo.7693963>.

Author contributions

CK: Conceptualization, Data curation, Formal Analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review and editing. MM: Conceptualization, Data curation, Formal Analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Software, Supervision, Validation, Visualization, Writing – original draft, Writing – review and editing. SA: Writing – original draft, Writing – review and editing.

Funding

The author(s) declared that financial support was not received for this work and/or its publication.

References

- Ajakwe, S. O., and Kim, D.-S. (2024). Facets of security and safety problems and paradigms for smart aerial mobility and intelligent logistics. *IET Intell. Transp. Syst.* 18, 2827–2855. doi: 10.1049/itr2.12579
- Ajakwe, S. O., and Kim, D.-S. (2025). Eqai: explainable quantum-empowered antispooofing intelligence for trustworthy connected autonomous vehicles communication. *IEEE Internet Things J.* doi: 10.1109/JIOT.2025.3638378
- Bello, R. J. (2020). On layer normalization in the transformer architecture. *Proc. ICML*, 765–773.
- Bojarski, M. (2016). End to end learning for self-driving cars. *arXiv Preprint arXiv: 1604.07316*.
- Bouthillier, X., Laurent, C., and Vincent, P. (2019). Unreproducible research is reproducible. *Proc. ICML*, 725–734.
- Caruana, R. (1997). Multitask learning. *Mach. Learn.* 28 (1), 41–75. doi: 10.1023/a:1007379606734
- Chattopadhyay, A., Sarkar, A., Howlader, P., and Balasubramanian, V. N. (2018). Grad-CAM++: generalized gradient-based visual explanations. *Proc. IEEE WACV*, 839–847.
- Chefer, H., Gur, S., and Wolf, L. (2021). Transformer interpretability beyond attention visualization. *Proc. IEEE/CVF CVPR*, 782–791.
- Dosovitskiy, A. (2021). An image is worth 16x16 words: transformers for image recognition at scale. *Proc. ICLR*.
- Gao, X., Wang, Z., Feng, Y., Ma, L., Chen, Z., and Xu, B. (2023). “Benchmarking robustness of AI-enabled multi-sensor fusion systems: challenges and opportunities,” in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)* (New York, NY, United States: Association for Computing Machinery), 871–882. doi: 10.1145/3611643.3616278
- Gildenblat and contributors (2021). pytorch-grad-cam. *GitHub*.
- Gunning, D., Stefik, M., Choi, J., Miller, T., Stumpf, S., and Yang, G. Z. (2019). XAI-Explainable artificial intelligence. *Sci. Robotics* 4 (37), eaay7120. doi: 10.1126/scirobotics.aay7120
- He, H., and Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Trans. Knowl. Data Eng.* 21 (9), 1263–1284.
- Kendall, Y. G., and Cipolla, R. (2018). Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. *Proc. IEEE/CVF CVPR*, 7482–7491.

Acknowledgments

The authors thank the maintainers of the PULP-DroNet v3 dataset and the developers of the timm and pytorch-grad-cam open-source libraries, whose work made this study possible.

Conflict of interest

The author(s) declared that this work was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Generative AI statement

The author(s) declared that generative AI was not used in the creation of this manuscript.

Any alternative text (alt text) provided alongside figures in this article has been generated by Frontiers with the support of artificial intelligence and reasonable efforts have been made to ensure accuracy, including review by the authors wherever possible. If you identify any issues, please contact us.

Publisher's note

All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

Supplementary material

The Supplementary Material for this article can be found online at: <https://www.frontiersin.org/articles/10.3389/ffutr.2026.1923890/full#supplementary-material>

- Kolesnikov, A. (2020). Big transfer (BiT): general visual representation learning. *Proc. ECCV*, 491–507.
- Lamberti, L., Bellone, L., Benini, L., and Palossi, D. (2022). *PULP-DroNet v3 Dataset*. Geneva, Switzerland: Zenodo. doi: 10.5281/zenodo.7693963
- LeCun, Y., Muller, U., Ben, J., Cosatto, E., and Flepp, B. (2005). Off-road obstacle avoidance through end-to-end learning. *Proc. NeurIPS* 18, 739–746.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollar, P. (2017). Focal loss for dense object detection. *Proc. IEEE ICCV*, 2980–2988.
- Loshchilov, A., and Hutter, F. (2019). Decoupled weight decay regularization. *Proc. ICLR*.
- Palossi, D., Loquercio, A., Conti, F., Flamand, E., Scaramuzza, D., and Benini, L. (2019). A 64-mW DNN-based visual navigation engine for autonomous nano-drones. *IEEE Internet Things J.* 6 (5), 8357–8371. doi: 10.1109/jiot.2019.2917066
- Samek, W., Binder, A., Montavon, G., Lapuschkin, S., and Muller, K. R. (2017). Evaluating the visualization of what a deep neural network has learned. *IEEE Trans. Neural Netw. Learn. Syst.* 28 (11), 2660–2673. doi: 10.1109/TNNLS.2016.2599820
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-CAM: visual explanations from deep networks via gradient-based localization. *Proc. IEEE ICCV*, 618–626.
- Singla, S. B., and Kaur, A. (2021). Survey on applications of UAVs in precision agriculture. *J. Ambient. Intell. Humaniz. Comput.* 12, 10329–10342.
- Thrun, S., Burgard, W., and Fox, D. (2005). *Probabilistic Robotics*. Cambridge, MA: MIT Press.
- Tobin, R. F., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. *Proc. IEEE/RSJ IROS*, 23–30.
- Touvron, H. (2021). Training data-efficient image transformers and distillation through attention. *Proc. ICML*, 10347–10357.
- Wightman, R. (2019). PyTorch image models (timm). *GitHub*.
- Zhang, H., Cisse, M., Dauphin, Y. N., and Lopez-Paz, D. (2018). Mixup: beyond empirical risk minimization. *Proc. ICLR*.